Problem J. Trajan 算法

请回忆用于计算无向图中割点（articulation points，又称 cut vertices）的 *Tarjan* 算法，其伪代码如左侧所示。

Algorithm 1 The Correct Tarjan Algorithm

```
1: function TARJAN(i, d)
2:     visited[i] ← true
3:     depth[i] ← d
4:     low[i] ← d
5:     childCount ← 0
6:     isArticulation ← false
7:
8:     for each j in adj[i] do
9:         if j = parent[i] then
10:            continue
11:        end if
12:        if not visited[j] then
13:            parent[j] ← i
14:            TARJAN(j, d + 1)
15:            childCount ← childCount + 1
16:            if low[j] ≥ depth[i] then
17:                isArticulation ← true
18:            end if
19:            low[i] ← min(low[i], low[j])
20:        else
21:            low[i] ← min(low[i], depth[j])
22:        end if
23:    end for
24:    if (parent[i] ≠ null and isArticulation)
       or (parent[i] = null and childCount > 1)
       then
25:        Output i as articulation point
26:    end if
27: end function
```

Algorithm 2 The Incorrect Trajan Algorithm

```
1: function TRAJAN(i, d)
2:     visited[i] ← true
3:     depth[i] ← d
4:     low[i] ← d
5:     childCount ← 0
6:     isArticulation ← false
7:
8:     for each j in adj[i] do
9:         if j = parent[i] then
10:            continue
11:        end if
12:        if not visited[j] then
13:            parent[j] ← i
14:            TRAJAN(j, d + 1)
15:            childCount ← childCount + 1
16:            if low[j] ≥ depth[i] then
17:                isArticulation ← true
18:            end if
19:        end if
20:        low[i] ← min(low[i], low[j])
21:
22:
23:    end for
24:    if (parent[i] ≠ null and isArticulation)
       or (parent[i] = null and childCount > 1)
       then
25:        Output i as articulation point
26:    end if
27: end function
```

在伪代码中，$adj[i]$ 是包含所有与节点 i 相邻的节点的数组，*parent* 是一个元素初始值均为 null 的数组，*visited* 是一个元素初始值均为 false 的数组。

一些学生可能很难理解这个算法，特别是从第 19 行到第 22 行。他们无法理解为什么 *Tarjan* 算法需要以不同的方式计算 $low[i]$ 的值，所以他们可能会记住一个错误版本的算法。我们称这个错误版本的算法为 *Trajan* 算法，如右侧所示。注意，*Tarjan* 和 *Trajan* 算法几乎相同，除了函数名称和第 19 行到第 22 行的这 4 行以外。

给定一个包含 n 个节点和 m 条边的无向连通图，对于每个节点 u，问是否存在一个节点 v 和一个二维数组 adj（adj 必须表示给定的图），使得 TARJAN$(v, 0)$ 和 TRAJAN$(v, 0)$ 在节点 u 上产生不同的结果（也就是说，*Tarjan* 算法认为顶点 u 是一个割点，而 *Trajan* 算法认为不是，反之亦然）。

Input

有多组测试数据。输入的第一行包含一个整数 T（$1 \le T \le 10^4$）表示测试数据组数，对于每组测试数据：

第一行输入两个整数 n 和 m（$2 \le n \le 10^5$，$1 \le m \le 10^5$），表示给定图中的节点和边的数量。

对于接下来 m 行，第 i 行输入两个整数 u_i 和 v_i $(1 \le u_i, v_i \le n)$，表示图中连接节点 u_i 和 v_i 的一条边。

保证给定的图是连通的，并且没有自环或重边。另外，所有数据的 n 之和以及 m 之和不超过 10^6。

Output

每组数据输出一行，包含一些由单个空格分隔的整数，并按升序排列。每个整数是 *Tarjan* 和 *Trajan* 算法可能在其上产生不同结果的节点。如果没有这样的节点，则输出一行 `Empty`。

Example

standard input	standard output
2 9 12 1 3 2 3 1 2 3 6 6 9 9 3 4 6 5 6 4 5 7 9 8 9 7 8 2 1 1 2	3 6 9 Empty

Note

对于第一组样例数据：

- 对于 $u = 3$ 考虑 $v = 1$ 和 $adj = [[3, 2], [3, 1], [1, 2, 6, 9], [6, 5], [6, 4], [4, 5, 3, 9], [9, 8], [9, 7], [7, 8, 3, 6]]$。
- 对于 $u = 6$ 考虑 $v = 4$ 和 $adj = [[2, 3], [1, 3], [9, 6, 2, 1], [6, 5], [6, 4], [4, 5, 3, 9], [9, 8], [9, 7], [7, 8, 3, 6]]$。
- 对于 $u = 9$ 考虑 $v = 7$ 和 $adj = [[2, 3], [1, 3], [9, 6, 2, 1], [5, 6], [4, 6], [9, 3, 5, 4], [9, 8], [9, 7], [7, 8, 3, 6]]$。

注意，所有的 adj 在解释中都是 1-索引的。